

UNIVERSIDAD DEL VALLE DE GUATEMALA

CC3067 Redes

Kevin Velásquez



Proyecto #1

Uso de un protocolo existente

Ihan Marroquín - 23108

GUATEMALA, 20 de septiembre del 2026

8. Especificación del servidor MCP desarrollado

Para el proyecto desarrollé un servidor MCP orientado al mantenimiento de una planta de bebidas. La idea es que el chatbot pueda consultar máquinas, repuestos y órdenes de trabajo sin darle acceso directo al archivo donde se guardan los datos. El modelo solamente decide qué herramienta necesita; el servidor valida los argumentos y realiza la operación.

8.1 Implementación general

El protocolo se programó manualmente con JSON-RPC 2.0 y la biblioteca estándar de Python. No utilicé FastMCP ni otro SDK de MCP. La misma clase de servicio se usa en las dos versiones del servidor, por lo que las herramientas producen el mismo resultado sin importar si la conexión es local o remota.

Elemento	Especificación
Nombre	industrial-maintenance-server
Versión del servidor	2.0.0
Versión MCP	2025-11-25
Formato	JSON-RPC 2.0 codificado en UTF-8
Servidor local	stdio; un mensaje JSON por línea
Servidor remoto	Streamable HTTP; mensajes JSON en /mcp
Persistencia	Archivo JSON con escritura atómica y bloqueo entre hilos

8.2 Endpoints de la versión remota

Método	Ruta	Uso
GET	/health	Comprueba si el contenedor está activo.
POST	/mcp	Recibe solicitudes y notificaciones JSON-RPC.
DELETE	/mcp	Cierra la sesión indicada en MCP-Session-Id.
GET	/mcp	Devuelve 405 porque no se usa un flujo SSE independiente.

La inicialización responde con un identificador en el encabezado MCP-Session-Id. Después de eso, el cliente debe enviar ese valor junto con MCP-Protocol-Version: 2025-11-25. El endpoint remoto también acepta Authorization: Bearer con el token configurado en MCP_AUTH_TOKEN.

8.3 Ciclo de comunicación

1. El cliente envía initialize con la versión de protocolo y sus capacidades.
2. El servidor responde con la versión aceptada, sus capacidades y la información del servidor.
3. El cliente envía notifications/initialized. Esta notificación no lleva id.
4. Después de la sincronización ya se pueden utilizar tools/list y tools/call.

Ejemplo reducido de la solicitud de inicialización:

```
{ "jsonrpc": "2.0", "id": 1, "method": "initialize",  
  "params": { "protocolVersion": "2025-11-25", "capabilities": {},  
    "clientInfo": { "name": "manual-client", "version": "2.0.0" } }
```

Si se intenta listar o ejecutar una herramienta antes de completar esos pasos, el servidor responde con el error -32002. Esto me ayudó a comprobar que el ciclo de vida no depende solamente de que el JSON tenga el formato correcto.

8.4 Herramientas disponibles

Herramienta	Parámetros	Resultado
list_machines	status (opcional)	Lista todas las máquinas o filtra por estado.
get_machine_status	machine_id	Devuelve el registro de una máquina.
list_work_orders	machine_id y status (opcionales)	Consulta las órdenes existentes.
check_spare_part	query	Busca un repuesto y calcula si hace falta reabastecer.
create_work_order	machine_id, issue, priority	Crea una orden nueva.
close_work_order	work_order_id, resolution	Cierra una orden abierta.

Las primeras cuatro herramientas son de lectura. `create_work_order` y `close_work_order` modifican el archivo de datos, así que el chatbot muestra el servidor, la herramienta y los argumentos antes de pedir confirmación al usuario.

8.5 Validaciones y manejo de errores

- Los parámetros adicionales y los valores fuera del esquema se rechazan antes de ejecutar la herramienta.
- Los identificadores de máquina u orden se comprueban contra los datos disponibles.
- El servidor HTTP limita el cuerpo a 1 MiB y exige contenido JSON.
- Los orígenes de navegador se comparan con `MCP_ALLOWED_ORIGINS` para reducir el riesgo de DNS rebinding.
- Las claves y el token se cargan desde variables de entorno y no se guardan en el repositorio.

Cuando una herramienta falla, el servidor conserva el id de la solicitud y devuelve un error JSON-RPC o un resultado de herramienta con `isError`. De esta manera el cliente puede diferenciar un error del protocolo, un error de argumentos y un problema propio de la operación.

8.6 Ejemplo de uso y comprobación

Una consulta de estado se envía de la siguiente forma:

```
{"jsonrpc": "2.0", "id": 3, "method": "tools/call",  
  "params": {"name": "get_machine_status",  
    "arguments": {"machine_id": "SELL-01"}}}
```

La respuesta conserva id: 3 y coloca la información de SELL-01 dentro de `result.structuredContent`. Probé el servicio, el ciclo MCP, stdio, HTTP, sesiones, validación de Origin y el cliente remoto con la suite incluida en tests. El resultado fue 12 pruebas ejecutadas y 12 correctas.

```
Ran 12 tests in 2.492s  
OK
```

9. Análisis de comunicación con Wireshark

Para analizar el contenido sin exponer un token real utilicé la versión HTTP en 127.0.0.1:8000. La captura se hace sobre el adaptador de loopback con el filtro tcp port 8000 y luego se ejecuta scripts.verify_remote. El programa produce siempre la misma secuencia, por lo que es sencillo relacionar cada POST con su respuesta.

También se puede hacer una segunda captura hacia Cloud Run en el puerto 443. En ese caso se observan DNS, TCP y TLS, pero el cuerpo JSON-RPC queda cifrado. Por eso separé la evidencia de contenido local de la evidencia del transporte remoto.

9.1 Mensajes observados

Orden	Mensaje	Tipo	Respuesta
1	initialize, id 1	Sincronización y solicitud	HTTP 200, InitializeResult e identificador de sesión
2	notifications/initialized	Sincronización y notificación	HTTP 202; no hay respuesta JSON-RPC
3	tools/list, id 2	Solicitud	HTTP 200 con las seis herramientas
4	tools/call, id 3	Solicitud	HTTP 200 con el resultado de list_machines
5	DELETE /mcp	Control del transporte	HTTP 204 y cierre de la sesión

Las solicitudes y respuestas se relacionan por el campo id. La notificación notifications/initialized no tiene id, entonces no recibe una respuesta JSON-RPC. El código 202 solamente indica que HTTP aceptó la notificación; no debe interpretarse como un objeto result.

9.2 Capa de enlace

En la prueba local los dos procesos se encuentran en la misma computadora. Por eso la captura no pasa por una trama Ethernet física ni necesita ARP para llegar a otro equipo. Wireshark puede mostrar un encabezado de loopback en lugar de Ethernet II. Cuando la prueba se hace contra Cloud Run, la MAC destino que aparece pertenece al gateway de la red local, no al servidor de Google. Las direcciones MAC cambian en cada salto.

Los datos útiles para esta capa son el tipo de enlace, la longitud de la trama y, cuando sí hay Ethernet, las MAC de origen y destino junto con el EtherType.

9.3 Capa de red

IP identifica los extremos lógicos. En la captura local aparecen direcciones loopback; en la remota, el nombre de Cloud Run se resuelve por DNS y el tráfico se dirige a una IP pública. El encabezado permite revisar la versión de IP, origen, destino, TTL o Hop Limit y el valor que identifica a TCP. Si existe NAT, la dirección privada del equipo puede cambiar al salir a Internet.

9.4 Capa de transporte

TCP establece una conexión entre un puerto efímero del cliente y el puerto 8000 de la prueba local, o el 443 de la prueba remota. Al comienzo se observa el intercambio SYN, SYN-ACK y ACK. Los números de secuencia y acuse mantienen el orden, mientras que la ventana controla cuántos datos pueden enviarse sin confirmación. El filtro tcp.analysis.flags permite revisar retransmisiones o segmentos fuera de orden.

9.5 Capa de aplicación

HTTP transporta cada mensaje MCP dentro de un POST a /mcp. El cuerpo utiliza application/json y el encabezado Accept permite application/json y text/event-stream. Después de initialize, el identificador de sesión y la versión MCP se envían en las siguientes solicitudes. JSON-RPC define campos como jsonrpc, id, method, params, result y error; MCP define qué significan initialize, tools/list y tools/call.

La captura deja claro que MCP no sustituye a TCP ni a HTTP. Es la conversación de aplicación que viaja sobre esas capas. En la conexión remota TLS se coloca entre HTTP y TCP para cifrar el token y los mensajes. Es normal que Wireshark no muestre el JSON sin las claves de sesión.

9.6 Filtros utilizados

Propósito	Filtro de Wireshark
Conversación local completa	tcp.port == 8000
Solicitudes POST	http.request.method == "POST"
Respuestas HTTP	http.response.code
Inicio y cierre TCP	tcp.flags.syn == 1 tcp.flags.fin == 1 tcp.flags.reset == 1
Posibles anomalías	tcp.analysis.flags
Conexión remota cifrada	tls tcp.port == 443

El procedimiento completo quedó en docs/WIRESHARK_ANALYSIS.md. Allí también se explica cómo usar Follow TCP Stream para reconstruir los mensajes y qué campos señalar en cada capa.

10. Conclusiones y comentario sobre el proyecto

10.1 Conclusiones

MCP separa la decisión del modelo de la ejecución. El LLM puede proponer una herramienta y sus argumentos, pero el servidor sigue siendo responsable de validar los datos y decidir si la operación puede realizarse. Esta separación fue especialmente importante en las herramientas que crean o cierran órdenes.

La implementación manual me permitió entender la diferencia entre una solicitud, una respuesta y una notificación. El caso más claro fue notifications/initialized: HTTP devuelve 202, pero a nivel de JSON-RPC no existe una respuesta porque el mensaje no tiene id.

También comprobé que la lógica de las herramientas puede compartirse entre stdio y HTTP. Lo que cambia es el transporte. En HTTP tuve que agregar sesiones, versión de protocolo, validación de Origin, autenticación y un cierre explícito, cosas que no aparecen de la misma forma cuando el servidor es un subprocesso local.

Wireshark ayudó a relacionar el programa con las capas vistas en clase. Las direcciones de enlace solo sirven para el siguiente salto, IP identifica los extremos lógicos, TCP entrega los datos de forma ordenada y HTTP transporta los objetos JSON-RPC. MCP trabaja encima de todo eso y define el orden y significado de la conversación.

10.2 Dificultades

La parte que más me costó fue mantener el mismo ciclo MCP en dos transportes. Con stdio el proceso permanece abierto y la secuencia es directa. En HTTP cada mensaje llega en una solicitud separada, por lo que fue necesario guardar el estado de la sesión y rechazar mensajes que llegaran antes de terminar la inicialización.

En la captura también fue fácil confundir el HTTP 202 con una respuesta del protocolo y pensar que la MAC observada en una prueba remota pertenecía a Cloud Run. Revisar cada capa por separado evitó esas dos interpretaciones.

10.3 Comentario personal

Antes del proyecto veía MCP principalmente como una forma de conectar funciones con un chatbot. Después de programarlo desde cero entendí que hay una negociación, capacidades, esquemas, ids, sesiones y errores que deben respetarse. Un SDK resuelve gran parte de ese trabajo, pero hacerlo manualmente hizo visible lo que realmente ocurre entre cliente y servidor.

La combinación de pruebas y registros fue muy útil. Pude revisar el mismo mensaje en el log, en el cliente y en Wireshark sin gastar créditos de la API. Para un sistema real todavía haría falta una base de datos, autenticación administrada y auditoría, pero la versión del proyecto permite demostrar el protocolo y comparar claramente el servidor local con el remoto.

Referencias

1. Model Context Protocol. Especificación 2025-11-25.
<https://modelcontextprotocol.io/specification/2025-11-25>
2. JSON-RPC Working Group. JSON-RPC 2.0 Specification.
<https://www.jsonrpc.org/specification>
3. Wireshark Foundation. Wireshark User's Guide.
https://www.wireshark.org/docs/wsug_html_chunked/
4. Repositorio del proyecto. https://github.com/Ihan-Marroquin/Proyecto1_redes